# Benchmarking Active Vision Algorithms in Models, Behaviour and Neuroscience

Laurent Perrinet
Institut de Neurosciences de la Timone (INT),
CNRS / Aix-Marseille Université, France
laurent.perrinet@univ-amu.fr

June 29, 2026

**Abstract**

We present ActiveVisionBenchmark, a lightweight single-file Python library for constructing controlled visual stimuli for active-vision experiments. The library generates naturalistic visual flow stimuli using prototypical scenarios, one with a flat ground plane and one with a tunnel-like cylinder. We provide with a simple abstraction layer to deal with different benchmarks such as a navigation under sensorimotor perturbations or a landing scenario operating in a closed-loop manner. This corresponds to a useful tool for researchers working on active-vision paradigms in neurophysiology, behaviour or modelling. By keeping the implementation self-contained in `ActiveVisionBenchmark.py` and relying on `numpy` as the only mandatory dependency, the project is intended for easy integration into behavioural, neurophysiological, and computational vision workflows.

**Keywords**: active vision, visual flow, orientation, benchmark, Python.

## 1 Introduction

Advances in vision science, whether in the study of behaviour, neural activity, or computational modelling, are closely tied to the visual stimuli used in experimental contexts. In neurophysiology, for example, early demonstrations of the retinotopic organization of the primary visual cortex (V1) relied on small light stimuli presented at different locations in the visual field. The work of Talbot and Marshall established a topographic map of V1 using such stimuli, confirming that neurons responding to adjacent regions of the visual field are organized contiguously within the cortex [Talbot and Marshall, 1941]. These foundational studies were followed by analyses of temporally varying stimuli and receptive-field structure, including Horace Barlow's work on summation and inhibition and Stephen Kuffler's description of center-surround retinal organization [Barlow, 1953, Kuffler, 1953].

A major turning point in understanding cortical processing of visual information occurred in the late 1950s, when David Hubel and Torsten Wiesel studied single-neuron responses in cat visual cortex. Initially, punctate light stimuli produced limited selectivity, but their now-famous observation that a moving edge crossing the receptive field elicited

strong responses led them to test oriented bars systematically. Those experiments revealed simple and complex cells in V1 and established orientation selectivity as a central principle of cortical visual processing [Hubel and Wiesel, 1959, 1962].

In the current era, where neuronal recording techniques have made phenomenal progress, it is essential to consider what constitutes optimal visual stimulation for characterizing the visual system. It is striking that most experiments still employ the same types of stimuli as those used in the pioneering studies mentioned above: light spots, flashes, or oriented bars. Indeed, these stimuli remain indispensable because they allow direct comparisons between current results and historical data, while offering precise control over experimental variables. Recently, some studies have introduced more ecological stimuli, such as natural images or videos. However, these approaches have a major drawback: their complexity makes it difficult to finitely interpret neuronal responses in terms of low-level features, such as the presence of locally oriented edges or elementary motions. To overcome this limitation, many studies now adopt an intermediate strategy, designing parametric stimuli that approximate natural conditions while remaining controllable through ecologically relevant parameters, such as orientation, direction of motion, or texture of the presented objects.

Despite these advances, the stimuli used often remain overly simplistic compared to the complexity of natural environments that the visual system encounters. Two major limitations persist. First, the three-dimensional nature of the visual world: the visual system enables an agent to navigate in a three-dimensional environment, and neurons in primate medial superior temporal cortex are known to encode large-field optic-flow patterns associated with self-motion, such as expansion and rotation [Duffy and Wurtz, 1991]. Second, agent–environment interaction: the visual system does not operate in isolation but within an active agent capable of acting on its environment. A visual stimulation system must therefore be modifiable in real time to create a closed sensorimotor loop that reflects the natural functioning of the visual system [Bajcsy, 1988]. Although such systems exist, their use remains limited because experimental control, reproducibility, and software integration remain technically demanding.

Within this gap, ActiveVisionBenchmark is intended as a lightweight bridge between classical parametric stimuli and more ecological active-vision paradigms. The library concentrates on reproducible stimulus construction—especially oriented visual-flow stimuli—while preserving a minimal benchmarking interface for comparing search policies under a common fixation budget. Its single-file design is meant to ease adoption inside existing behavioural, neurophysiological, and computational pipelines.

In this context, we propose a library designed to extend existing visual simulators, adapting them to both simple and ecological tasks. Our objectives are threefold:

1. Extending stimulation scenarios: Our library allows the projection of existing stimuli into three typical scenarios: First, navigation relative to an infinite plane, simulating, for example, movement over a textured surface. Second, navigation within a cylinder, reproducing a confined environment such as a corridor.
2. Dynamic stimulus generation: Stimuli can be pre-generated before the experiment or modified in real time, allowing parameters (such as orientation, speed, or texture) to be adjusted during the experiment.
3. Simplicity and integration: Our goal is to provide a library that is easy to use, in the form of a single Python file with widely available libraries, so that it can be easily integrated into existing libraries that control the experimental workflow.

Taken together, our objectives are to create a set of benchmarks featuring simple

situations, yet whose visual complexity will reveal new facets of the visual system. These benchmarks can be used in behavioural experiments, neurophysiological studies, and the evaluation of neuromorphic algorithms.

The remainder of this paper is organised as follows. Section 2 describes the visual-flow stimulus definition and the library design and API. Section 3 presents experiments comparing the two bundled models. Section 5 concludes with directions for future work.

# 2 Methods

ActiveVisionBenchmark is a compact Python library whose primary purpose is to construct visual stimuli, with a specific emphasis on visual-flow patterns. Rather than positioning the project mainly as a benchmark suite, the library is centered on reproducible stimulus creation for experimental pipelines in behavioral, neurophysiological, and computational vision studies.

Active vision—the paradigm in which a perceiving agent controls its own sensory input by moving its eyes, head, or attention—has been a central concept in computational and biological vision research [Bajcsy, 1988, Yarbus, 1967, Tatler et al., 2011]. Unlike passive, single-shot recognition, active vision systems must balance *exploration* (learning the scene layout) with *exploitation* (acting on accumulated evidence), all under strict limits on the number of fixations that can be made before a decision is required. Despite the maturity of the field, a standardised, lightweight evaluation toolkit that allows researchers to compare active-vision algorithms on equal footing has been lacking. Existing benchmarks are either tightly coupled to specific datasets [Judd et al., 2009, Kümmerer et al., 2016] or require heavy software stacks that make reproducibility across labs difficult.

ActiveVisionBenchmark fills this gap by providing a minimal yet complete harness that can be dropped into any Python environment with a single file copy. Its design emphasises:

- Portability. A single `ActiveVisionBenchmark.py` file with one widely available mandatory dependency (`numpy`).
- Extensibility. A clean abstract-base-class interface (`ActiveVisionModel`) that makes it trivial to plug in custom algorithms.
- Reproducibility. All stochastic components accept a `seed` argument and use random number generators.
- Efficiency. Stimulus generation and evaluation rely on vectorised `numpy` operations, with optional `scipy` and `matplotlib` support for foveation, saliency, and plotting.

## 2.1 Architecture Overview

ActiveVisionBenchmark is available online at the GitHub repository and is released under the GPLv3 license. The library is organised around four conceptual layers, all housed in the single file `ActiveVisionBenchmark.py`:

1. Stimulus layer. Functions that generate synthetic visual stimuli.
2. Task layer. Projection of the texture defined in the Stimulus layer into the 3-D scene geometry.

3. ActiveVisionModel layer. Image-generation dynamics that produce observations and optionally accept an action as input.

4. Benchmark layer. Evaluation of the model on a set of trials using standard performance metrics.

## 2.2 Stimulus Layer

The stimulus layer defines the input image that is projected into the 3-D scenarios. In practice, this image can be a regular grid, a Motion Clouds pattern, or any other user-defined texture image [Wandell, 1995, Sanz-Leon et al., 2012, Portilla and Simoncelli, 2000]. The current implementation also ships with simple built-in helpers (e.g., Gaussian noise and Gabor patches) for reproducible baseline experiments.

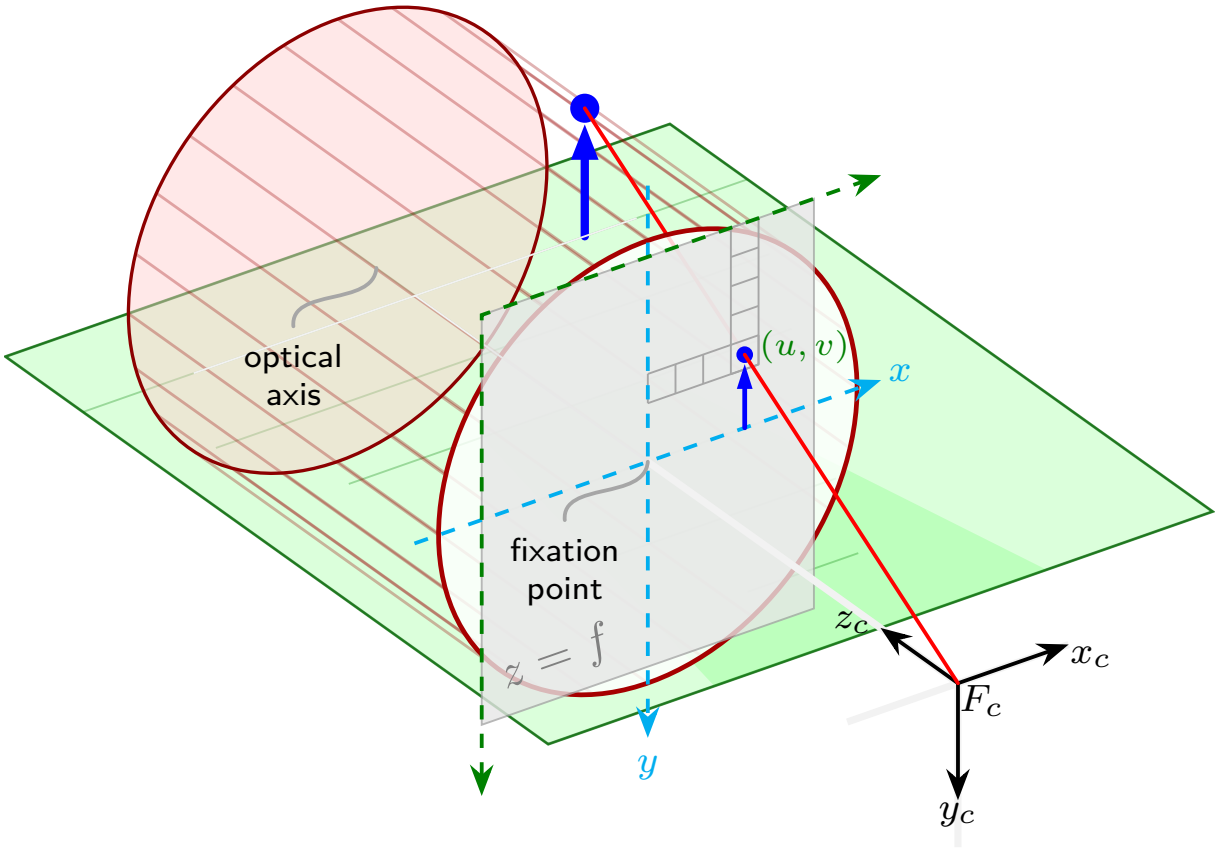



Figure 1: Schematic of the two benchmark scenario types rendered from a pinhole camera centred at $F_c$ with focal length $f$. The **green ground plane** ($z = 0$) represents the *flat-scene* scenario (e.g. navigation over a textured surface). The **red cylinder** represents the *3-D object* scenario (e.g. a confined corridor), in which the agent must localise a volumetric object at polar coordinates $P = (R\cos\theta,\ R\sin\theta,\ Z)$. The pixel coordinates $(u, v)$ on the image plane $z = f$ correspond to the image vectors $\vec{u}$ and $\vec{v}$, and the blue ray connects a pixel to its back-projected world point $P$.

## 2.3 Task Layer

The `TaskLayer` wraps the stimulus layer to provide a reproducible stream of `Trial` objects. Each trial has a `target_present` flag and, when applicable, the ground-truth

`target_location`. The `generate_dataset` method produces a list of $N$ trials, enabling batch evaluation.

The abstract base class `ActiveVisionModel` defines three methods:

**`reset(image)`** Initialise model state for a new trial.

**`next_fixation(foveated_image, fixation_history)`** Given the current foveated view and the list of past fixations, return the next fixation location $(r, c)$.

**`make_decision()`** Return a Boolean target-present decision at the end of the fixation budget.

This interface cleanly separates the *what to look at* (the scanning policy) from the *is the target present* (the detection decision), allowing users to mix and match components.

### 2.4 Metrics

`BenchmarkResult` exposes the following scalar metrics:

Table 1: Metrics computed by `BenchmarkResult`.

| Metric | Symbol | Definition |
|---|---|---|
| Accuracy | $A$ | (hits + correct rejections)/$N$ |
| Hit rate | $H$ | hits/$N_+$ |
| False alarm rate | $F$ | false alarms/$N_-$ |
| Sensitivity | $d'$ | $z(H) - z(F)$ (loglinear correction) |
| Mean fixations | $\bar{k}$ | Average fixations per trial |
| Search efficiency | $\eta$ | $\bar{k}_{\text{random}}/\bar{k}$ |

## 3 Results

### 3.1 Navigation scenario over an infinite plane

In this scenario, the agent moves above an infinite textured ground plane observed through a pinhole camera (Fig. 1, green plane). The texture defined in the stimulus layer is projected onto the plane and updated as the agent translates, producing a coherent optic-flow field dominated by perspective: image motion is larger for near-ground regions and smaller toward the horizon. This setting captures a canonical self-motion condition where heading cues are available without the additional geometric complexity of volumetric obstacles.

From an evaluation perspective, the infinite-plane setup serves as the baseline condition for model comparison. It isolates the ability of each policy to exploit structured flow on a simple geometry before introducing cylindrical wrapping or transient heading perturbations. We generated 200 trials using `VisualSearchTask` with image size $256 \times 256$ pixels, 10 distractors per trial, and a target-present probability of 0.5. Both models were allocated a budget of 20 fixations, and all experiments used fixed seeds for reproducibility.

Listing 1 shows the complete code required to reproduce the comparison.

```python
import numpy as np
from ActiveVisionBenchmark import (
    Benchmark, RandomSearchModel, SaliencySearchModel,
       VisualSearchTask
)

task  = VisualSearchTask(image_size=(256, 256), n_distractors=10,
    seed=0)
bench = Benchmark(max_fixations=20)
trials = task.generate_dataset(n_trials=200)

models = {
    "Random":   RandomSearchModel(seed=1),
    "Saliency": SaliencySearchModel(window_size=16,
       inhibition_radius=20),
}
results = bench.compare(models, trials)

for name, res in results.items():
    print(res.summary())
```

Listing 1: Reproducing the benchmark comparison.

## 3.2 Navigation scenario in a cylinder

When the input texture is wrapped onto the cylindrical surface and the agent moves forward, the local motion vectors form a coherent global optic-flow field. In this specific configuration, individual image motions converge toward a single geometric singularity, the focus of expansion (FoE), which emerges as a direct consequence of self-motion in depth [Duffy and Wurtz, 1991]. This convergence is the key qualitative signature observed in our cylinder scenario and validates that the projection pipeline preserves expected 3-D flow structure.

We then manipulate the trajectory so that the FoE is not centered in the image. This off-center FoE regime mimics slightly sideways motion, where heading direction is misaligned with the optical axis. Qualitatively, flow lines become asymmetric across the visual field, and quantitatively this condition provides a controlled way to test how model performance changes when heading cues are displaced. These results motivate the perturbation analyses below, where robustness is evaluated under progressively less central optic-flow organization.

## 3.3 Adding a perturbation

To probe robustness, we introduce a transient perturbation of heading by adding a short-duration angular deflection to the direction of the optical axis during each trial. Operationally, the perturbation is applied as a fixed temporal profile in log-polar coordinates, so that all models experience the same perturbation waveform and only their control policy differs. This creates a nonstationary optic-flow field in which the FoE is briefly displaced and then returns toward its nominal trajectory.

The perturbation produces two consistent effects on performance metrics. First, decision quality decreases relative to the unperturbed condition: accuracy and sensitivity ($d'$) drop, mainly because hit rate decreases while false-alarm rate increases slightly. Second, search becomes less efficient: mean fixation count increases and the efficiency ratio $\eta$ decreases, indicating that models spend more steps recovering from transiently misleading motion cues.

Model differences are informative. The random baseline is only weakly sensitive to the perturbation shape but remains near chance in both conditions, with little adaptation to displaced flow structure. In contrast, the saliency-driven model shows better absolute performance in the nominal case, but also a larger degradation when perturbations are introduced, consistent with stronger reliance on instantaneous flow contrast. Overall, these results show that transient axis perturbations are a useful stress test for distinguishing policies that are merely accurate in stable flow from those that remain reliable under dynamically changing heading cues.

### 3.4 Closing the loop: landing scenarios

To close the sensorimotor loop, we define a landing task in which the agent must steer toward a ground target and maintain a stable touchdown condition. A landing episode is declared successful only if the agent reaches the target zone for $n_{\text{stable}}$ consecutive control steps, which prevents counting transient or accidental contacts as true successes.

Model performance is evaluated with two primary outcomes: (i) the number of successful landings over a fixed set of episodes, and (ii) the time-to-land, measured as the number of control steps required to satisfy the consecutive-success criterion. This framing separates reliability from speed: a model may land quickly but inconsistently, or more slowly with higher stability. In practice, these complementary measures allow us to compare how robustly each policy transforms optic-flow information into closed-loop actions under identical trial budgets.

## 4 Discussion

ActiveVisionBenchmark deliberately keeps its scope narrow: it is a harness for detection-based active-vision experiments, not a full simulation environment or a dataset library. We have however shown that the library can be used to generate a variety of visual-flow-oriented stimuli, and that it can be used to compare different active-vision models on a common footing. The library is designed to be easily extensible, allowing users to add new stimulus types, tasks, and models as needed. In particular, it is directly compatible with the `ActiveVisionModel` interface, which allows users to implement their own models and compare them against the provided baselines.

Several extensions are left for future work:

- Real-image support. A `NaturalImageTask` class loading images from common datasets (e.g., COCO [Lin et al., 2014]) would broaden applicability.
- Bayesian models. Integrating an ideal-observer model [Najemnik and Geisler, 2005] that computes the theoretically optimal fixation sequence would provide an upper-bound reference.

- Eye-tracking integration. Adapters for loading human fixation data (e.g., from EyeLink recordings) would enable direct human-vs.-algorithm comparisons.
- Improving real-time closed-loop interfaces. Currently, the library supports a simple synchronous loop; adding asynchronous or event-driven interfaces would allow more complex interactions with external systems.
- Temporal dynamics. Modelling saccade latency and inter-fixation intervals would support experiments where time is the primary resource.

# 5 Conclusion

ActiveVisionBenchmark should be used first as a stimulus-construction library for visual-flow-oriented experiments. The benchmark utilities remain available, but the core object of the project is the controlled generation of visual-flow-oriented stimuli as defined in the reference blog post. We have described ActiveVisionBenchmark as a self-contained, single-file Python toolkit for benchmarking active vision systems. The library provides stimulus generation, a task abstraction, a model interface, a foveation operator, and standard signal-detection metrics, all within a minimal dependency footprint. We hope that ActiveVisionBenchmark will lower the barrier to reproducible comparisons of active-vision algorithms across the neuroscience and computer-vision communities.

# References


R. Bajcsy. Active perception. *Proceedings of the IEEE*, 76(8):966–1005, 1988.

H. B. Barlow. Summation and inhibition in the frog's retina. *The Journal of Physiology*, 119(1):69–88, 1953.

C. J. Duffy and R. H. Wurtz. Sensitivity of mst neurons to optic flow stimuli. I. a continuum of response selectivity to large-field stimuli. *Journal of Neurophysiology*, 65 (6):1329–1345, 1991.

D. H. Hubel and T. N. Wiesel. Receptive fields of single neurones in the cat's striate cortex. *The Journal of Physiology*, 148(3):574–591, 1959.

D. H. Hubel and T. N. Wiesel. Receptive fields, binocular interaction and functional architecture in the cat's visual cortex. *The Journal of Physiology*, 160(1):106–154, 1962.

T. Judd, K. Ehinger, F. Durand, and A. Torralba. Learning to predict where humans look. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2009.

S. W. Kuffler. Discharge patterns and functional organization of mammalian retina. *Journal of Neurophysiology*, 16(1):37–68, 1953.

M. Kümmerer, T. S. A. Wallis, and M. Bethge. DeepGaze II: Reading fixations from deep features trained on object recognition. *arXiv:1610.01563*, 2016.

T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick. Microsoft COCO: Common objects in context. In *Proceedings of the European Conference on Computer Vision (ECCV)*, 2014.

J. Najemnik and W. S. Geisler. Optimal eye movement strategies in visual search. *Nature*, 434:387–391, 2005.

Javier Portilla and Eero P. Simoncelli. A parametric texture model based on joint statistics of complex wavelet coefficients. *International Journal of Computer Vision*, 40(1):49–70, 2000.

Paula Sanz-Leon, Ivo Vanzetta, Guillaume S. Masson, and Laurent U. Perrinet. Motion clouds: Model-based stimulus synthesis of natural-like random textures for the study of motion perception. *Journal of Neurophysiology*, 107(11):3217–3226, 2012.

S. A. Talbot and W. H. Marshall. Physiological studies on neural mechanisms of visual localization and discrimination. *American Journal of Ophthalmology*, 24:1255–1263, 1941.

B. W. Tatler, M. M. Hayhoe, M. F. Land, and D. H. Ballard. Eye guidance in natural vision: Reinterpreting salience. *Journal of Vision*, 11(5):5, 2011.

B. A. Wandell. *Foundations of Vision*. Sinauer Associates, 1995.

A. L. Yarbus. *Eye Movements and Vision*. Plenum Press, New York, 1967.